

Time Series and Systematic Trading

Two Weeks at Imperial

Jonty Ali · 6 August 2026 · Notes · ~20 min

Two modules at Imperial College London’s **Summer School on Machine Learning, Applied Statistics, and Quantitative Finance**. The first: [Machine Learning and Statistics for Time Series Analysis](#), a “comprehensive exploration of probability theory and time series analysis”. And the second: [Systematic Trading](#), which introduces “the design and deployment of systematic trading strategies used by hedge funds and other ‘smart money’”.

These are notes on some of what I learned, module by module, with the key points & equations, shallow but broad technical coverage.

Reader should be comfortable with basic probability and a bit of undergraduate statistics. I skip the foundations and start from stationarity.

Contents

1	Module One: Modern Time Series Analysis	2
1.1	Making a sequence tractable	2
1.2	Seeing what’s underneath	2
1.3	The model itself	3
1.4	Using the model	4
2	Module Two: From Signals to Trading	4
2.1	A good signal alone isn’t enough	4
2.2	Portfolio construction	5
2.3	Newer approaches	5
2.4	Risk	5
3	Big Point on Both Modules	6
4	Notes on the Course Itself	6

1 Module One: Modern Time Series Analysis

Making a sequence tractable

Say you have one observed time series e.g. a stock price, a temperature record, and you want to say something statistical about it. Normally, statistics wants many repeated observations of the same random thing. Here you only ever get one path, one series over time. So how do you estimate anything at all?

The answer is to assume the process behaves the same way at every point in time, we say its average value isn't drifting, and that how two points' *covariance* (how much they tend to move together) only depends on how far apart they are, not on which time these two points you picked are at. This assumption is called **weak stationarity**, and it's what lets you treat different stretches of a single series as if they were separate samples from the same underlying process.

That's what finally lets us estimate real quantities from a single path: a mean, a variance, an autocorrelation.

Normally, computing the covariance between two points in a series would need both time indices: how X at time 5 relates to X at time 8 is potentially different from how it relates to X at time 30. Under stationarity, all that matters is the gap between them, called the **lag**, h . That gap-dependent covariance is the **autocovariance function**, written $\gamma(h)$: the covariance between a point in the series and the point h steps later. Dividing by the variance $\gamma(0)$ normalises this into the **autocorrelation function** $\rho(h)$, a number between -1 and 1 that's always exactly 1 at lag 0 , since a point matches itself perfectly.

$$\gamma(h) = \text{Cov}(X_{t+h}, X_t) \qquad \rho(h) = \frac{\gamma(h)}{\gamma(0)}$$

Seeing what's underneath

Let us look at a simple and ubiquitously important time series model, **white noise**: a sequence of values that are completely independent of each other, with constant mean and variance, no trend, and no relationship between one point and the next.

Every diagnostic check later in this piece is really asking the same question: does this *look like* white noise, or is there structure left that our model should capture?

Practically, this means plotting the data first (trends and outliers are sometimes obvious just by eye) then dealing with any trend or seasonality before modelling anything, and finally checking afterwards that nothing structural got left behind in the residuals, which is our "does this look like white noise".

Removing a trend is usually done by **differencing**: instead of modelling the series itself, you model the change from one point to the next. A straight-line trend has the same slope everywhere, so subtracting each value from the one before it cancels that slope out completely. One round is sometimes not enough, for example a curved trend, in which case you could difference multiple

times.

$$\nabla X_t = X_t - X_{t-1}$$

the lag-1 difference, removes a linear trend

Seasonality works the same way, just at a different gap: instead of comparing each point to the one right before it, you compare it to the one a full cycle earlier e.g. 12 months back for a yearly pattern. If the cycle repeats reliably, that difference cancels it out for the same reason lag-1 differencing cancels a trend: whatever was consistent from cycle to cycle gets subtracted away.

The model itself

Once a series is stationary, you need an actual model for it: some process whose autocorrelation function (ACF) matches the shape you're seeing. Two simple models are very useful and real models often combine or build upon them.

The first is the **first order autoregressive** model, AR(1): today's value is some fraction of the previous value, plus fresh noise.

$$X_t = \phi X_{t-1} + \varepsilon_t$$

ε_t is white noise; ϕ is some parameter; $|\phi| < 1$ for a well-behaved process

The second is the **first order moving average** model, MA(1): today's value is just this period's noise plus a fraction of *last period's* noise. There's no dependence on past *values*, only on past *shocks*.

$$X_t = \varepsilon_t + \theta \varepsilon_{t-1}$$

ε_t is white noise; θ is some parameter

An **ARMA(p, q)** process is more general and combines both ideas at once, p autoregressive terms and q moving-average terms. AR(1) is really just ARMA(1, 0) and MA(1) is just ARMA(0, 1). If the raw series wasn't stationary to begin with, you difference it d times first (as in the last section) and fit ARMA to what's left. That's called **ARIMA(p, d, q)**, an even more general model with one extra parameter for how much differencing it took. So, similarly, **ARMA(p, q)** is just **ARIMA(p, 0, q)**.

Fitting one of these to real data means finding the parameters (ϕ 's and θ 's). Typically by **maximum likelihood**, we find the parameters that make the data you actually observed as likely as possible under the model.

Choosing p and q is a separate problem from fitting them: p is how many past values you include, q is how many past shocks (noise), so a bigger p means more history feeding into today's prediction, a bigger q means more past noise terms doing the same.

A bigger model can always fit the training data at least as well, so "best fit" alone isn't a fair way to choose, you need something that penalises unnecessary complexity too. For example, that's what **AIC** and **BIC** do: each is a standard way of trading fit against model size, so the order you

end up with is justified by the data rather than just whichever model had the most flexibility to bend to the data exactly.

Once a model is fitted, you still need to check it actually worked: whether what's left over in the residuals is genuinely just noise, or whether structure was missed, you also don't want to overfit your model. That's all a topic of its own, one I won't get into during this overview, but relevant tests to look up are **Box-Pierce and Ljung-Box**.

Using the model

Once a model is fitted and checked, the point of it is usually to look forward. Given everything observed up to now, what's the best guess for the value h timesteps from now?

The idea is simple: you already know the observed values and the fitted φ 's and θ 's, so you just apply the model's own equation forward, step by step, using your forecasts in place of the (unknown) future values as you go.

$$\hat{X}_{t+h} = \varphi \hat{X}_{t+h-1} + \dots$$

each forecast feeds into the next, further out, using the same recursion the model was fitted on

A single number isn't the whole story though. Push the recursion far enough and the forecast stops changing, since each new step depends more on your *previous forecast* than on the actual data, and less and less of the original observed values are left in the calculation, the prediction just tends toward the series' long run average. It gets less certain the further out you go, too: more unknown noise terms have had a chance to accumulate in between, so the honest range of "where this could actually land" (the **prediction interval**) widens the further out you look.

2 Module Two: From Signals to Trading

A good signal alone isn't enough

Finding a signal that predicts something is only the start of the hard work. Testing a strategy on past data can make it look reliably and even massively profitable, but that same strategy often loses money once it's actually being traded. The main reason is that we have not considered **trading costs**, for example: brokers charge a fee just to place the trade; buying and selling prices are never quite the same (the **spread**), so simply crossing from one to the other costs you something; and your own order can move the price against you before it's even filled.

There's another hurdle on top of all this, even once a strategy is genuinely profitable after costs. Money isn't free: you could always just hold cash, or something close to it, and earn the **risk-free rate** for doing nothing at all. A strategy only really adds value if it beats that baseline, so the real bar isn't "is this profitable," it's "is this profitable by more than doing nothing would have been." That makes an already hard problem harder still: it's not enough for a signal to survive costs, it has to survive costs *and* clear a return that was available for free.

Portfolio construction

If you maximised for *only* expected return, we would only care about the one strategy with the highest return, but the standard way to think about this is a trade-off between return and risk: for a given amount of risk you're willing to take, what mix of positions gives you the highest expected return? You might expect this just means picking whichever single strategy has the best return per unit of risk and ignoring the rest, but combining strategies averages returns without averaging risk in the same way. As long as two strategies aren't highly correlated, some of their ups and downs cancel out when held together, so a portfolio of several decent strategies can end up with far less risk than any single one of them alone. Spreading weight across several imperfect, weakly-correlated strategies can raise your return per unit of risk even if none of them individually is the top performer.

A common and well-known metric used for this is the **Sharpe ratio**: your return above the risk-free rate, divided by your returns' standard deviation. A high Sharpe ratio means you're being well paid for the risk you're taking, not just making money by taking on a lot of it.

$$\text{Sharpe} = \frac{R - R_f}{\sigma}$$

R is your return, R_f the risk-free rate, σ the standard deviation of your returns

The **Markowitz portfolio** is the specific mix of weights that gets you the best possible trade-off between return and risk, the highest Sharpe ratio achievable from your set of strategies. In practice, though, this solution is notoriously unstable, since it's built from *estimated* returns and correlations rather than the true ones, and small errors in those estimates can propagate wildly into your final "optimal" weights.

Newer approaches

Everything so far assumes you already have a signal to trade and a sensible way to size it. Recent work applies **machine learning** to both halves of that problem: finding signals in the first place, and deciding how to size and combine them once you have several.

Reinforcement learning has also been used to treat execution itself as a control problem rather than a prediction problem, learning how to actually place trades over time rather than just what to trade. These are each substantial topics of their own, and ones I'd like to properly explore separately, rather than compress into a paragraph here.

Risk

A strategy is only as good as your understanding of how much you could lose. We touched on risk in **Portfolio construction**, but there's a lot more to it than a single standard deviation.

Two common and simple ways to measure it are **Value at Risk (VaR)** and **Expected Shortfall (ES)**. VaR asks: what's the worst loss I'd expect at some chosen confidence level, say 95% of the time? It tells you where that threshold sits, but nothing about how bad things get if you're unlucky enough to cross it. VaR also has an awkward mathematical property: combining two risky posi-

tions can sometimes look riskier under VaR than holding either one alone, the opposite of what diversification is supposed to achieve.

Expected Shortfall fixes both problems at once. It's the average loss *given* that you've crossed the VaR threshold, which gives an honest picture of the tail rather than just its edge, and it doesn't suffer from VaR's diversification quirk either. That combination is why ES is generally considered the more modern, transparent choice when presenting a risk model.

Both are usually estimated the simple way: a single standard deviation from historical returns, treated as roughly constant over time. But in practice, markets don't sit at one constant level of risk, they move between calmer and more volatile periods, and a risk estimate built this way has no way to notice volatility rising before a bad move actually happens. Getting this right, so the model actually adapts as volatility changes, is its own significant topic.

3 Big Point on Both Modules

The thing that surprised me was how often both modules came back to the same underlying question: not *"does this model fit?"*, but *"is there still structure it has failed to capture?"* Whether looking at ARMA residuals or evaluating a trading strategy's risk model, you're constantly asking whether what's left behind really behaves the way your assumptions say it should.

I also expected most of the work to be finding and fitting models. Instead, a surprisingly large part is spent questioning whether the model is actually valid. Are the residuals really white noise? Have you overfit the data? Are the assumptions you've made actually justified? Much of modelling turns out to be less about building models than about trying to prove your own model wrong.

4 Notes on the Course Itself

This isn't everything the two modules covered, it's a selection of what I found most worth explaining, kept broad and shallow on purpose. There was also some fundamentals on statistics and regression provided, so the course is more accessible than this writeup may make it out.

There was a practical side to both modules, the first in R and the second in Python (though no extensive knowledge is needed). This breaks up the fast theory heavy lectures and lets the ideas sink in.

Also, the course actually runs three modules total, not just the two I did; what I've called Module One and Module Two here are really the course's Module Two and Module Three, renamed for simplicity in this writeup.

There's also a social side to it I haven't mentioned: you meet interesting people from a lot of different backgrounds/universities all interesting in similar topics, and each module included a social event to the museums in Kensington. Worth knowing about if you're considering applying yourself.

Everything above is my own writing and reflection on what I learned attending this in 2026, not official material from Imperial or notes directly from the course. The actual content of the summer school is also not fixed, it's run at the discretion of the Imperial lecturers may well change between years.
